

Project 68: Audio Keyword Spotting

Goal: Create a model or tool that:

1. Listens to short audio clips (or real-time input)
 2. Detects if a **specific keyword** is spoken
 3. Outputs the detected word or triggers an action
-

Common Datasets

- **Google Speech Commands** (35+ words): [Download](#)
 - Words include: yes , no , up , down , go , stop , etc.
-

Step-by-Step Implementation Using Google Speech Commands + CNN

Step 1: Install Required Libraries

```
pip install torchaudio torch torchvision matplotlib
```

Step 2: Load Dataset and Preprocess

```
import torchaudio
from torchaudio.datasets import import SPEECHCOMMANDS
import os
```

```
class SubsetSC(SPEECHCOMMANDS):
    def __init__(self, subset):
        super().__init__("./", download=True)
        self._walker = [w for w in self._walker if subset in w]

train_set = SubsetSC("training")
test_set = SubsetSC("testing")
```

Step 3: Create Spectrogram Input

```
import torch
from torchaudio.transforms import MelSpectrogram

transform = MelSpectrogram(sample_rate=16000, n_mels=64)

def preprocess(waveform):
    return transform(waveform).squeeze(0)
```

Step 4: Define a CNN for Classification

```
import torch.nn as nn

class KeywordCNN(nn.Module):
    def __init__(self, num_classes):
        super().__init__()
        self.conv = nn.Sequential(
            nn.Conv2d(1, 8, kernel_size=3),
            nn.ReLU(),
            nn.MaxPool2d(2),
            nn.Conv2d(8, 16, kernel_size=3),
            nn.ReLU(),
            nn.MaxPool2d(2)
        )
        self.fc = nn.Sequential(
```

```

        nn.Linear(16 * 15 * 15, 128),
        nn.ReLU(),
        nn.Linear(128, num_classes)
    )

    def forward(self, x):
        x = self.conv(x.unsqueeze(1))
        x = x.view(x.size(0), -1)
        return self.fc(x)

```

Step 5: Train the Model

```

# Prepare DataLoader (example using small set)
labels = sorted(list(set(dat[2] for dat in train_set)))
label_to_index = {label: i for i, label in enumerate(labels)}

def collate_fn(batch):
    xs, ys = [], []
    for waveform, _, label, *_ in batch:
        xs.append(preprocess(waveform))
        ys.append(label_to_index[label])
    return torch.stack(xs), torch.tensor(ys)

from torch.utils.data import DataLoader

train_loader = DataLoader(train_set, batch_size=32, collate_fn=collate_fn)
test_loader = DataLoader(test_set, batch_size=32, collate_fn=collate_fn)

model = KeywordCNN(num_classes=len(labels))
optimizer = torch.optim.Adam(model.parameters(), lr=0.001)
loss_fn = nn.CrossEntropyLoss()

for epoch in range(5):
    model.train()
    for xb, yb in train_loader:
        preds = model(xb)
        loss = loss_fn(preds, yb)
        optimizer.zero_grad()
        loss.backward()

```

```
optimizer.step()  
print(f"Epoch {epoch+1} done.")
```

Step 6: Predict Keywords

```
def predict_keyword(wav_path):  
    waveform, sr = torchaudio.load(wav_path)  
    mel = preprocess(waveform)  
    pred = model(mel.unsqueeze(0))  
    predicted_label = labels[torch.argmax(pred)]  
    return predicted_label  
  
print(predict_keyword("yes.wav"))
```

Key Concepts

- **Keyword Spotting:** Classifies audio clips into pre-defined keywords
 - **Spectrograms:** Visual audio representations for CNNs
 - **Trigger word detection:** Foundation of wake-word systems
-

Project Summary

- **Input:** Short audio clip (1 sec)
- **Output:** Detected keyword
- **Model:** CNN trained on Mel-spectrograms

- **Use Case:** Voice commands, wake words, interaction cues